Chapter 7. Designing lock-free concurrent data structures

This chapter covers

- Implementations of data structures designed for concurrency without using locks
- Techniques for managing memory in lock-free data structures
- Simple guidelines to aid in the writing of lock-free data structures

In the last chapter we looked at general aspects of designing data structures for concurrency, with guidelines for thinking about the design to ensure they're safe. We then examined several common data structures and looked at example implementations that used mutexes and locks to protect the shared data. The first couple of examples used one mutex to protect the entire data structure, but later ones used more than one to protect various smaller parts of the data structure and allow greater levels of concurrency in accesses to the data structure.

Mutexes are powerful mechanisms for ensuring that multiple threads can safely access a data structure without encountering race conditions or broken invariants. It's also relatively straightforward to reason about the behavior of code that uses them: either the code has the lock on the mutex protecting the data or it doesn't. However, it's not all a bed of roses; you saw in chapter 3 how the incorrect use of locks can lead to deadlock, and you've just seen with the lock-based queue and lookup table examples how the granularity of locking can affect the potential for true concurrency. If you can write data structures that are safe for concurrent access without locks, there's the potential to avoid these problems. Such a data structure is called a *lock-free* data structure.

In this chapter we'll look at how the memory-ordering properties of the atomic operations introduced in chapter 5 can be used to build lock-free